Contract-Centered Engineering

Under Collapsed Implementation Costs

v2.16

Gregory Tomlinson

Contract-Centered Engineering

Implementation Cost and the Shift in Engineering Topology

For most of modern software engineering, implementation cost shaped practice.

Writing a production-grade system required sustained effort: design, implementation, review, integration, testing, deployment, and iteration. Producing even a single complete implementation demanded coordination across roles. Producing multiple independent implementations of the same system was rarely practical outside regulated or safety-critical domains.

Under those constraints, engineering converged around a single artifact. The codebase was not only the system; it functioned as its authority. Tests were written against it. Documentation was derived from it. Disagreements were resolved by it. Rewrites were avoided because they were destabilizing and expensive.

That arrangement was economically rational.

When implementation is costly, authority concentrates in the implementation because replacing it is prohibitive.

Generative systems alter that condition.

Independent implementations can now be produced rapidly at negligible marginal cost. Alternative interpretations of the same operational description can be generated, compared, discarded, and regenerated without the coordination overhead that previously constrained such experimentation.

When implementation cost collapses, engineering topology can change.

Authority no longer needs to reside in a particular implementation. A contract-centered topology becomes viable: a clause-structured operational contract defines system behavior; independent implementations are derived from it and evaluated against it. Implementations become interchangeable samples rather than singular authorities.

Scarcity relocates.

Implementation effort is no longer the dominant constraint. Specification precision is.

Invariants must be explicit. Authority boundaries must be defined. Scope constraints must be unambiguous. Deletion semantics must be articulated. Abort conditions must be encoded deliberately. These cannot be inferred reliably from vague intention. They must be specified.

The collapse of implementation cost does not eliminate discipline. It increases the leverage of specification.

Engineering shifts from defending a single artifact to stabilizing contracts that survive independent derivation.

Relationship to Prior Work

This work does not introduce contracts as a concept. It depends directly on the long-established tradition of contract-based specification, including Design by Contract (Meyer), formal specification practices, contract programming, and property-based testing traditions.

If invariants and explicit behavioral guarantees did not meaningfully constrain implementation, the method described in this paper would not function. The ability to evaluate independent derivations clause-by-clause presupposes that contracts are real constraints, not rhetorical artifacts.

Traditional Design by Contract embeds preconditions, postconditions, and invariants within a single implementation. Formal methods extend this tradition through mathematical specification and proof. Property-based testing evaluates a given implementation against generalized behavioral properties.

All of these approaches assume a primary implementation artifact that must be verified, tested, or proven.

The distinction here is economic rather than conceptual.

When independent implementations are expensive, contracts constrain a system. When independent implementations are inexpensive and repeatable, contracts can instead stabilize authority across multiple derivations.

The contribution of this work is not the invention of contracts, but the identification of a topology shift: collapsed implementation cost allows contracts to function as the stable artifact while implementations are sampled, compared, and refined against them.

This approach presumes the validity of contract-based specification. It explores what becomes possible once implementation is no longer the scarce resource.

The Contract-First Loop

A contract-centered topology requires a disciplined method. Reduced implementation cost does not reduce rigor; it changes where rigor applies.

The following seven-step loop specifies, derives, evaluates, and refines systems under conditions where independent implementation is inexpensive.

This is not a benchmark procedure. It is not a model ranking exercise. It is a method for stabilizing specifications through controlled independent derivation.

1. Contract-First Inversion

The traditional artifact order is inverted.

Instead of exploratory discussion producing code, exploratory discussion produces a clause-structured operational contract.

The contract defines operational semantics explicitly: identity rules, authority relationships, scope boundaries, deletion behavior, abort conditions, output guarantees, and invariant preservation requirements.

No implementation is written at this stage.

The contract is treated as the primary engineering artifact.

2. Adversarial Hardening

The drafted contract is not assumed correct.

Each clause is examined for interpretive slack, undefined terms, implicit assumptions, boundary ambiguity, and edge-case instability.

Ambiguity is treated as a defect.

Hardening proceeds in deliberate rounds. The objective is not theoretical completeness but reduction of foreseeable divergence under independent derivation.

3. Independent Derivation

The hardened contract is used to generate multiple independent implementations.

Independence is enforced. There is no shared memory across runs. There is no iterative correction between implementations. One derivation does not influence another.

Implementations may be generated by different AI systems, different configurations of the same system, human engineers, or external vendors.

Each implementation is a sample induced by the contract.

No implementation is authoritative.

4. Clause-Level Evaluation

The contract is decomposed into discrete evaluable clauses.

Clauses are categorized by role — safety-critical invariants (MUST), behavioral expectations (SHOULD), and structural requirements.

Each implementation is evaluated against each clause independently. Adherence is recorded explicitly.

Evaluation is contract-driven. The question is not stylistic preference but clause satisfaction.

The output is a clause-level adherence matrix.

5. Divergence Localization

Where deviations occur, they are traced to specific clauses.

Failures are categorized — authority substitution, scope expansion or contraction, invariant omission, deletion logic drift, schema variance, abort condition failure.

Patterns across implementations are analyzed.

The objective is to determine whether divergence originates from specification ambiguity or from independent misinterpretation of a clearly defined clause.

6. Clause Refinement

Only clauses associated with observed divergence are modified.

Refinement clarifies boundary semantics, converts implicit expectations into explicit invariants, tightens conditional language, and reduces interpretive flexibility.

Each modification is versioned and traceable to observed divergence.

The contract evolves in response to empirical evidence.

7. Re-Sampling and Convergence Measurement

The revised contract is used to generate new independent implementations.

Clause-level evaluation is repeated.

Convergence is measured as elimination of safety-critical failures, reduction in cross-implementation disagreement, and stabilization of defined behavior across independent derivations.

Convergence does not imply identical code. It implies behavioral stability under independent sampling.

Comparative Implementation Capability

The collapse of implementation cost makes comparative derivation practical.

Under prior constraints, producing multiple independent implementations required justification. Teams invested in a single artifact and iterated on it.

With generative systems, a single hardened contract can produce multiple independent implementations at negligible cost. These implementations may differ internally in structure or strategy, but they share a common origin.

This enables structured comparison.

Multiple implementations can be evaluated against the same clause structure. Adherence can be measured explicitly. Divergence can be categorized. Patterns can be observed.

This is not a leaderboard.

The purpose is not to determine which generative system is superior. The purpose is to expose instability in the specification and assess behavioral adherence under independent derivation.

When implementations are inexpensive, they are replaceable.

The contract remains stable while implementations are sampled, evaluated, discarded, or selected.

Contract as Cross-Functional Authority

Code expresses behavior precisely but is not broadly legible.

A clause-structured operational contract expresses behavioral guarantees and constraints explicitly.

It defines invariants that must always hold. It defines authority boundaries, scope constraints, deletion semantics, and abort conditions.

These constructs remain technical but are legible at the behavioral level across roles.

QA teams test against clauses. Product teams evaluate guarantees. On-call engineers reference explicit invariants during incident analysis. Change requests modify clauses rather than relying on institutional memory embedded in code.

Authority is decoupled from a particular implementation and anchored in specification.

Empirical Demonstration

The method was applied to a deterministic system specified as a clause-structured contract:

Artifact Sync System — Convergence Contract (v1.1)
(Published alongside this paper.)

The contract defined identity semantics, authority rules, deletion conditions, and convergence guarantees governing document synchronization.

It was hardened through multiple adversarial rounds prior to derivation.

Independent implementations were generated under enforced isolation, including at least one implementation produced by a different AI system than the one used to assist drafting.

Each implementation was evaluated clause-by-clause.

Clause Excerpt: Authority Rule

> “The document stored under the authoritative doc_id SHALL be treated as the single source of truth. Manual file variants SHALL NOT override the authoritative document unless explicitly promoted.”

Observed divergence included implicit merging of manual variants, filename-based identity inference, and conditional overrides without explicit promotion logic.

Clause Excerpt: Deletion Semantics

> “Manual file variants SHALL be removed when no authoritative document exists for the corresponding doc_id. Historical versions MAY be preserved but SHALL NOT participate in active synchronization.”

Observed divergence included unconditional deletion, indefinite retention of orphaned variants, and inconsistent deletion timing.

Clause Excerpt: Convergence Guarantee

> “Given identical authoritative state, independent nodes SHALL converge to identical active document sets after synchronization.”

Observed divergence included transient-state inconsistency and ambiguity in synchronization ordering.

Clause-Level Adherence Matrix (Excerpt)

Clause Category	Impl A	Impl B	Impl C
Authority Rule (MUST)	FAIL	PARTIAL	PASS
Deletion Semantics (MUST)	FAIL	PARTIAL	PASS
Convergence Guarantee (MUST)	PARTIAL	PASS	PASS
Schema Structure (STRUCTURAL)	PASS	PARTIAL	PASS

Divergence concentrated in authority and deletion clauses. Structural clauses were comparatively stable.

Clauses associated with recurrent divergence were refined. The contract was re-sampled. Subsequent derivations demonstrated reduced instability across previously unstable clauses.

The objective was not to rank systems. It was to test whether a clause-structured contract could remain behaviorally stable under independent derivation.

Divergence localized predictably and decreased under clause refinement.

A subsequent replication using stricter Convergence Contract v2.5.2 evaluated independent implementations across two AI systems under identical prompts. Both ultimately achieved full MUST-level compliance (16/16), though remediation rounds differed. The contract remained unchanged. Clause-level corrections reduced divergence until invariant stability was achieved.

Implications for Writing Software

If implementation can be produced in seconds, engineering effort concentrates where scarcity remains.

Code remains necessary. Architecture and performance remain technical disciplines.

But when implementations are inexpensive and replaceable, they need not carry authority.

The contract can.

Teams can agree on operational semantics before defending implementations. Competing implementations can be generated and evaluated without destabilizing system truth. QA tests against explicit clauses. On-call engineers reference defined invariants. Change requests modify the contract rather than relying on implicit behavioral memory.

This paper does not claim that contract-centered derivation guarantees superior software.

It claims something structural:

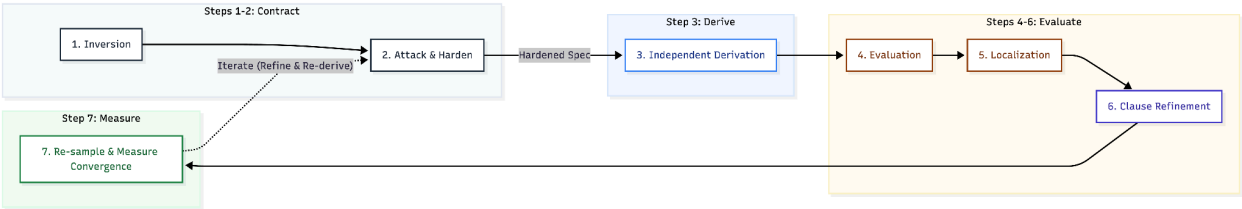
When implementation is inexpensive, separating specification authority from implementation artifact becomes practical.

That separation enables comparative derivation, measurable convergence, and cross-functional clarity in ways that were previously impractical.

Related Practitioner Writing

Independent practitioner writing reflects the same structural shift described in this paper: when implementation becomes inexpensive, precision in operational description becomes the constraining factor.

Oliver Cronk describes a renewed emphasis on specification clarity in generative-assisted development. Thoughtworks discusses the possibility that maintained specifications may become the durable artifact while code becomes transient.

Boris Tane documents approximately nine months of sustained specification-first generative workflow in practice, demonstrating that detailed operational descriptions can reliably drive implementation.

These accounts reflect the same structural condition described in this paper. The method presented here focuses on what becomes measurable when multiple independent implementations are derived from a shared contract and evaluated clause-by-clause.